

Hyper-Spherical Systems — About HypeS

What Is HypeS?

HypeS (Hyper-Spherical Systems) is a complete local AI execution environment built around a single radical idea: *every limitation of running large language models on consumer hardware is a solvable engineering problem.*

VRAM too small? HypeS streams model layers from NVMe through system RAM into GPU memory on demand — the model never knows the difference.

Cloud API costs too high? HypeS guarantees a **minimum** of 10× token compression before prompts leave your machine (often reaching 200–300:1 via substitution dictionaries), cutting token bills dramatically.

Models get repetitive or confused? HypeS monitors generation entropy in real time and autonomously applies a 5-step escalating recovery protocol.

Design Philosophy

1. Self-Contained

Every HypeS binary is a complete, standalone executable. There is no runtime to install, no Python environment to configure, no Docker container to manage. Drop the binary on any machine and run it.

2. Hardware-First

HypeS treats your hardware as a 4-tier memory hierarchy:

GPU VRAM (Hot) ← System RAM (Warm) ← NVMe SSD (Loaded / Ready-to-Fire) ← SATA HDD (Cold)

- **SATA HDD (Cold / Pre-flight):** Serves as the long-term storage and staging house for all models and intents.
- **NVMe SSD (Loaded / Hot):** Models are moved here and stacked in a non-Euclidean geometry to make them highly predictable and easy to stream. Crucially, a model is

only written to the NVMe **once**. It stays there until the session is closed, the system powers down, or space runs out (requiring replacement).

- **System RAM (Warm / Overflow):** Acts as a temporary overflow buffer. Data fires directly from the NVMe to the GPU VRAM, but if there is too much data attempting to load at once, it overflows into System RAM temporarily before piping right back into the VRAM (preventing VRAM thrashing).
- **GPU VRAM (Hot):** The ultimate destination for active model execution.

The *LayerIllusionist* engine ensures the model always sees a valid GPU pointer, while the *Markov-chain pattern predictor* watches which layers activate in sequence to prefetch data from NVMe/RAM precisely when needed, completely hiding latency.

3. Compression-Everywhere

Three independent compression layers work in parallel, featuring **sub-millisecond hardware-maintained compression and decompression**. This enables near-instantaneous processing with little to no code overhead doing the work:

Layer	What it compresses	Ratio
SISSI	Common LLM boilerplate phrases → §codes	2–4× tokens
CCTM Ultra	Semantic deduplication + differential encoding	10× Minimum (Often 200–300:1)
5+1 Homophonic Cipher	Character-level scramble using session-unique Unicode pool	Confidentiality

All three layers stack. The cloud model receives only the compressed + encrypted payload. Session key material is zeroed at teardown — the cloud cannot decode past sessions.

4. Living Models (SFS+)

Standard model files are static. SFS+ models are *alive*: - They borrow skills from peer models during a session - They permanently absorb capabilities where a peer demonstrably outperforms them - They retire their own outdated skills when superseded - They persist learned knowledge across all future sessions

The *Competitive Growth Engine* compares models domain by domain (Math, Code, Reasoning, Language, Creative, Science, Instruction, Safety) and makes transfer decisions autonomously according to a configurable learning policy.

5. Autonomous Self-Governance

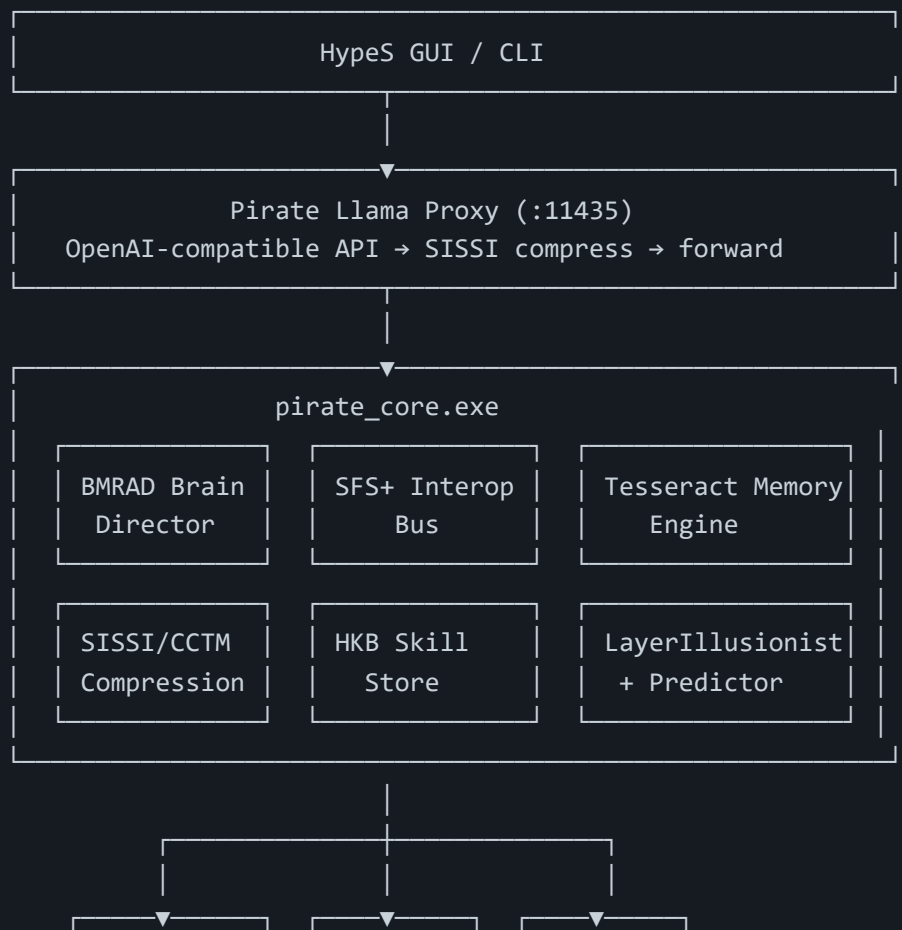
The *Brain Model Realignment & Attention Director (BMRAD)* is injected into every running model context. It:

- Profiles all attention heads by L1-norm importance
- Prunes heads scoring below threshold (max 15% per cycle)
- Detects generation loops via angular distance on the unit hypersphere
- Applies a 5-level escalating recovery (temperature → rephrase → consult → lateral thinking → escalate user)
- Requires 2-of-3 consensus (BMRAD + host model + peer jury) before committing any self-modification
- Monitors perplexity Z-scores to decide whether to route difficult queries to a better-equipped peer

6. Privacy by Default

- Zero telemetry until you explicitly opt in
- AES-256 encryption on all outbound payloads
- Ephemeral session ciphers — keys live in RAM only, zeroed at session end
- Proprietary source code — only compiled binaries are distributed

Architecture Overview



GPU VRAM (hot)	System RAM	NVMe (cold)
-------------------	---------------	----------------

Module Roster

#	Module	License Flag
1	Golden Candy Spinner — .hsc 4D vortex format	MODULE_GCS
2	BMRAD — Brain Director (AttentionAnalyzer, LoopDetector, ConsensusAuditor...)	MODULE_BMRAD
3	SFS/SFS+ Interop — CrossModel bus, CompetitiveGrowthEngine, ToolRegistry, Sandbox	MODULE_SFS_PLUS_INTEROP
4	Pirate Llama — Universal OpenAI-compatible proxy + Declutterizer	(base)
5	SISSI — Phrase codec, Handshake protocol, MNECP	MODULE_CIPHER
6	CCTM Ultra — 10× Cloud Token Compression	MODULE_CCTM_ULTRA
7	Tesseract Memory — LayerIllusionist, PatternPredictor, WeightStreamer, DraftTokenEngine	(base)
8	NVMe Benchmark — Sequential & random I/O characterization	(base)
9	HyperSphere Geometry — NeuronGraph, VirtualMoE, 4D coordinates	(base)
10	SFS Launcher + Onboarding — 7-step wizard, crypto key setup, drive selection	(base)
11	Universal Endpoint — 3-stage seal pipeline + AI asset auto-discovery	(base)
12	Cipher Engine — AES-256, Homophonic, Session Cipher, Argon2	MODULE_CIPHER
13	Python Bridge DLL — C ABI ctypes interface	(base)
14	4ID Avatar — Natural language generation, full-duplex voice, 4D animations	MODULE_4IDENTITY_AVATAR

#	Module	License Flag
15	HuggingFace Client — Model card fetch, recommendations	<i>(base)</i>
16	Thinstall Packager — Zero-install USB/NVMe portable bundle	MODULE_SFS_PLUS_INTEROP
17	Recursive Updater — Self-modification pipeline (MANUAL/SEMI/FULL autonomy)	MODULE_BMRAD
18	Recovery & Checkpointing — Pre-modification snapshots + PPL-triggered rollback	<i>(base)</i>
19	Telemetry & Session — Heartbeat, macro recorder, conversation manager	<i>(base)</i>
20	License Manager — Module bitmask, tier enforcement, pre-release gate	<i>(base)</i>

Developer & Contact

Developer: twiztedsocal

Project: Hyper-Spherical Systems

GitHub: github.com/thomasjosh1981/Hyper-Spherical-Systems

Hyper-Spherical Systems © 2026. Developed by twiztedsocal. Source code is proprietary and not publicly distributed.